Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Online

Faculty: Information Technology Honours - Data Science

Module Code: ITBDA4

Group: 1

Lecturer's Name: Mr. Gilbert Lekula

Student Full Name: Mckhyle Ganas

Student Number: EDUV12784928

Indicate	Yes	No
Plagiarism report attached		

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature	Date
Mckhyle.Ganas	2026/11/03

Lecturer's Comments:

Marks Awarded:	%

Signature	Date

Figure 1: Waterfall Workflow Chart 3
Figure 2: Africa Keyword Occurrences 5
Figure 3: Reducer of Keyword: Africa 6
Figure 4: Core-site.xml 7
Figure 5: Hdfs-site.xml 8
Figure 6: Mapred-site.xml 8
Figure 7: Yarn-site.xml 9
Figure 8: Command Prompt Verification of Hadoop 3.4.2 Configuration 9
Figure 9: Initial Dataset Loading 10
Figure 10: Data Frame of the documents 10
Figure 11: Top 5 words 12
Figure 12: Topic Probability 12
Figure 13: Topic 1 Bar Graph 13
Figure 14: Topic 2 Bar Graph 13
Figure 15: Topic 3 Bar Graph 13
Figure 16: Heatmap Topic Word Distribution 14
Figure 17: Quote and Author extraction 15
Figure 18: Data frame of Quotes and Authors 16
Figure 19: Cleaned Quote Column 17
Figure 20: Top Words for Summarisation 17
Figure 21: Frequency-Based Extractive Summarisation 18
Figure 22: TF-IDF Summarisation 18
Figure 23: Top 10 Word Frequency 19
Figure 24:Frequency Based and TF-IDF Summarisation Comparison 19
Figure 25: Extracted Book Data 21
Figure 26: Cleaned Extracted Book Data 22
Figure 27: SQL Table Creation and CVS Import 23
Figure 28: Confirming Book Records 23
Figure 29: Investigation 1- Overall Book Price Analysis 24
Figure 30: Investigation 2- Book Distribution by Price Range 24

Question 1:

1.1) Framework: Hadoop MapReduce:

Programming language: R

Input dataset: map_reduce_test_docs

Number of documents: 7 (doc1.txt – doc7.txt)

Search keyword: Africa/africa/AFRICA

Search condition: Case-insensitive

Required output: Document name, row number, and column number of each occurrence

Logical layout diagram of the MapReduce keyword search

2. The diagram below shows how the seven text documents are processed through the Hadoop MapReduce.

Waterfall Workflow Diagram of the Keyword Search Process

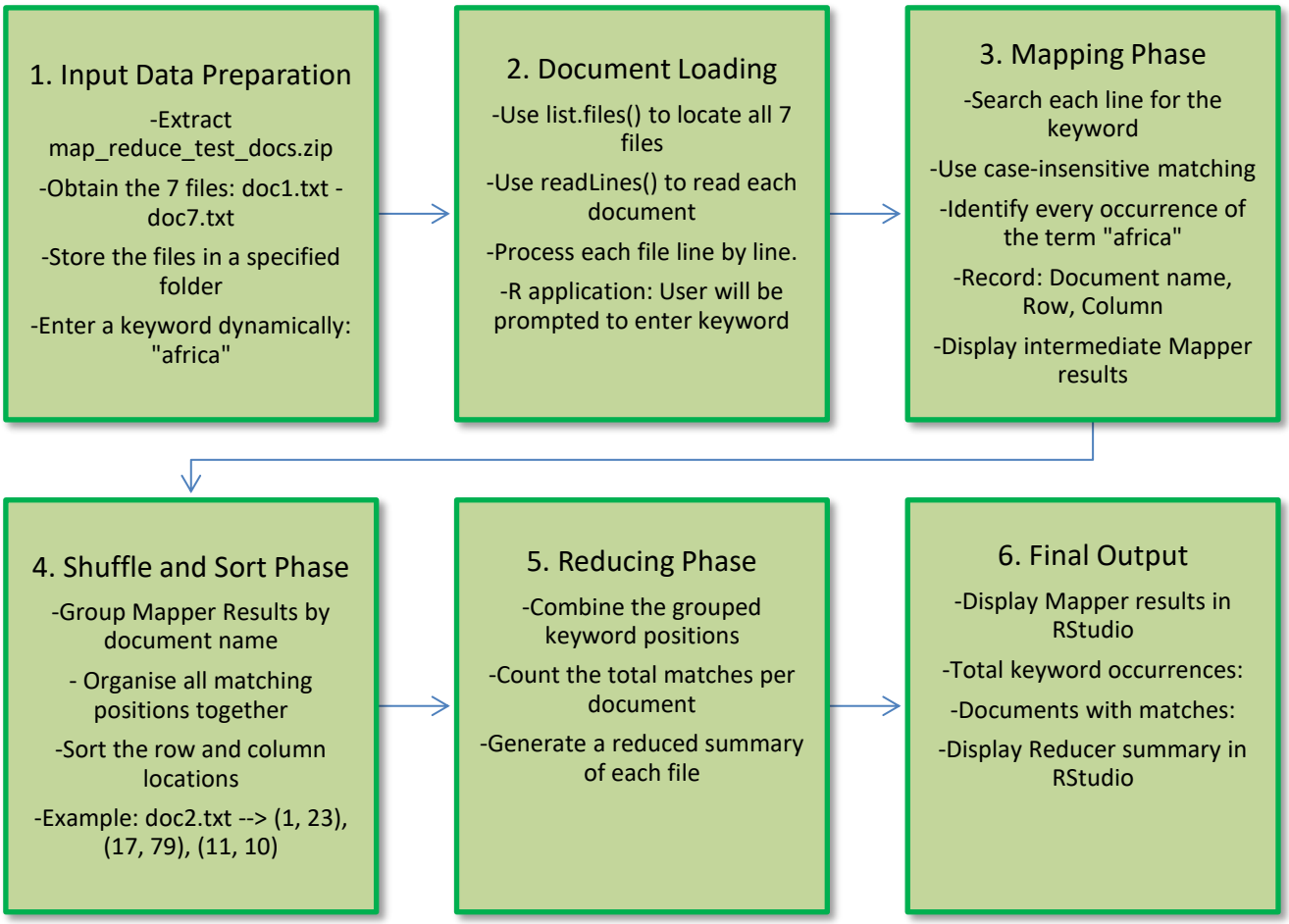


Figure 1: Waterfall Workflow Chart

3. A detailed explanation of the MapReduce process

Step 1: Input data preparation

The input dataset consists of seven text documents (doc1.txt-doc7.txt). Within a Hadoop environment, these documents would be stored in the Hadoop Distributed File System (HDFS) for distributed processing. For the R

implementation, the documents are accessed from a local directory (user workspace/device) using list.files() and readLines(). The keyword is dynamically entered by the user.

Step 2: Mapping phase

The Mapper's primary objective is for searching the desired keyword. Each Mapper performs the following operations:

-Search for every occurrence of the keyword "Africa/africa".

-And to record the document name, row number and starting column number. This will also generate key value pairs for each keyword match. For example, Key: doc1.txt Value: (2, 7). This means that the keyword appears in document 1, or row 2, starting at column 7.

The Mapper processes each document line by line and performs a case sensitive search for each specified keyword using gregexpr(). Where every keyword occurrence happens, an intermediate key value record is generated. The will contain the document name as the key, along with the keywords row and column positions as the value.

Step 3: Shuffle and sort phase

The intermediate records are then grouped according to their document names. In the R implementation, the split() function simulates the grouping operation which will allow for all the keyword locations belonging to the same document to be processed at once.

For example:

doc1.txt → [(2,7). (3,1)] | doc2.txt → [(1,14)]

This will allow the Reducer to process all matching positions associated with each document.

Step 4: Reducing phase

The Reducer processes each document group. Then sorts the matching locations by row and column, combines the positions into a readable format and then will calculate the total amount of keyword occurrences in each document.

Step 5: Final output

The final output will display the document name, the corresponding row and column positions and total keyword occurrences. Below is a dummy table for how the expected output could look:

Document	Row	Column
Doc1.txt	3	3
Doc2.txt	4	5
Doc2.txt	1	11
Doc4.txt	5	7
Doc7.txt	6	10

5. Important technical considerations

There are two details worth understanding should this assignment use R programming instead of Java. First, Hadoop's standard TextInputFormat normally provides the byte offset of a line and now its row number.

Therefore, in order to acquire accurate row numbers, the program would need to integrate suitable custom input reader that tracks line numbers or preprocess the files to their lines. Second, the column positions should be calculated regularly. An R implementation can utilise functions such as regexpr() or gregexpr() for locating each occurrence and add 1 to convert the zero-based index into a one based position.

1.2) Dynamic R implementation of the Mapper reduce (Code submitted)

1. Implementation Overview

The Mapper phase of the MapReduce framework was implemented using RStudio for searching across seven text documents (doc1.txt-doc7.txt) for a specified keyword. The objective was to identify every occurrence of the keyword and then return its document name, row number and starting column position.

The implementation supports case sensitive searching and allows the keyword to be entered dynamically through an RStudio input pop-up dialog box using rstudioapi::showPrompt(), rather than being hardcoded into the program. This also aligns with the assignments instructions to not have the word “Africa” not hardcoded.

2. Mapper Implementation and Processing

The implementation uses the following R functions:

1. list.files(): This will retrieve the seven text documents from the directory in which I stored the dataset.
2. readLines(): Reads each document line by line
3. gregexpr(): Identifies all the occurrences of the keyword, and using the ignore.case = TRUE for case insensitive matching.
4. seq_along(): This iterates through the lines for determining the row numbers
5. data.frame() and rbind(): This stores each matching occurrence with its document name, row number, and column.
6. lapply() Applies the Mapper function individually to all the seven documents.

3. Results and Interpretation

Using the dynamically entered keyword “africa”, the Mapper successfully processed all 7 documents and had identified 16 keyword occurrences across four documents.

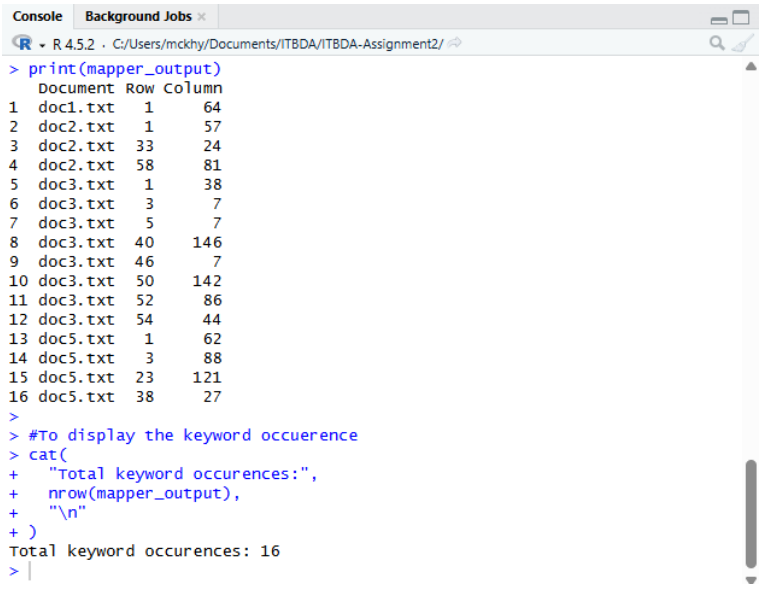


Figure 2: Africa Keyword Occurrences

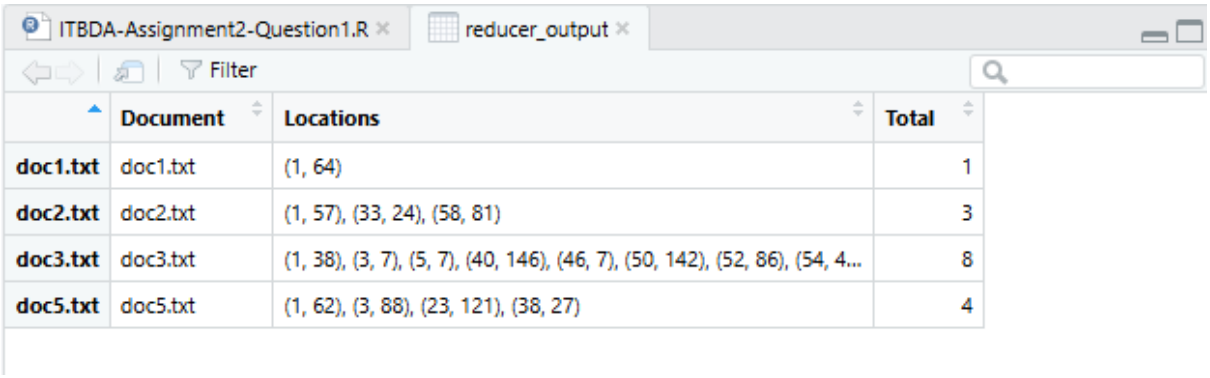
As per the results above, the highest number of occurrences was recorded in doc3.txt. There were no occurrences that were identified in doc4.txt, doc6.txt or doc7.txt. The Mapper also identified the location of each occurrence by returning the document, row number and column number of where the keyword was found. For example, in doc1.txt, the keyword was found in row 1, column 64. This indicates that it begins at the 64th character of the first line.

1.3) R Implementation of the Reducer

The Reducer phase was successfully implemented in RStudio to consolidate the key value records that was generated by the previous questions Mapper.

Firstly, I have utilised the split() function for grouping the Mapper results according to their document names, thus simulating the Shuffle and Sort Phase of the MapReduce framework.

A custom reducer() function was then formed to process each document group. The order() fuction arranged the keyword occurrences by row and column, while the paste0() combined their location into a readable format. Thereafter, the nrow() function calculated the total number of keyword occurrences per document. Finally, the lapply() function applied the Reducer to each document group, and the do.call(rbind …) function combined the results into a final table. The results of the keyword “africa” and its Reducer are shown below:


ITBDA-Assignment2-Question1.R × reducer_output ×

Filter

	Document	Locations	Total
doc1.txt	doc1.txt	(1, 64)	1
doc2.txt	doc2.txt	(1, 57), (33, 24), (58, 81)	3
doc3.txt	doc3.txt	(1, 38), (3, 7), (5, 7), (40, 146), (46, 7), (50, 142), (52, 86), (54, 4…	8
doc5.txt	doc5.txt	(1, 62), (3, 88), (23, 121), (38, 27)	4

Figure 3: Reducer of Keyword: Africa

As per the above results, doc3.txt contained the highest number of keyword occurrences, and the documents with no “africa” keywords were left out.

1.4)

Configuration File	Property	Value
Core-site.xml	Fs.defaultFS	Hdfs://localhost:9820
Hdfs-site.xml	Dfs.replication	1
Hdfs-site.xml	Dfs.namenode.name.dir	Local NameNode directory
Hdfs-site.xml	Dfs.datanode.data.dir	Local DataNode directory
Mapred-site.xml	Mapreduce.framework.name	Yarn
Yarn-site.xml	Yarn.nodemanager.aux-services	Mapreduce_shuffle

Implementation of Hadoop MapReduce Configuration Settings

Apache Hadoop version 3.4.2 was used to configure a single node environment for MapReduce processing. The implementation consisted of reviewing and configuring four XML files located within the Hadoop installation directory.

As per below table, the configuration settings are summarised:

The core-site.xml file specifies the default Hadoop Distributed File System (HDFS) address, while the hdfs-site.xml configures data replication and storage locations.

The mapred-site.xml file specifies yarn as the execution framework for MapReduce tasks. In addition to this, yarn-site.xml enables the shuffle service. This supports the transfer of data between Mapper and Reducer tasks. Overall, these settings provide the base for running MapReduce jobs within the Hadoop framework.

Below are screenshots of the practical evidence of the configuration settings:

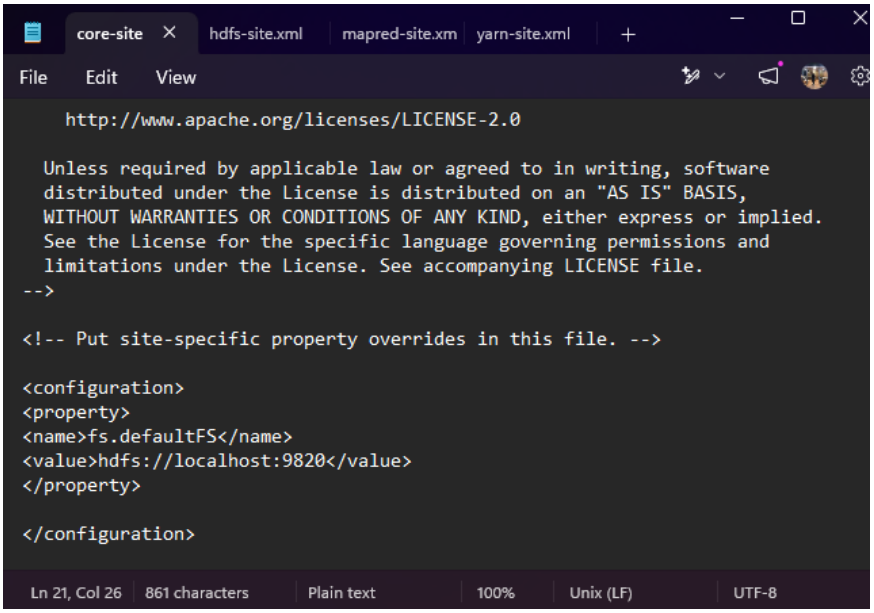


Figure 4: Core-site.xml

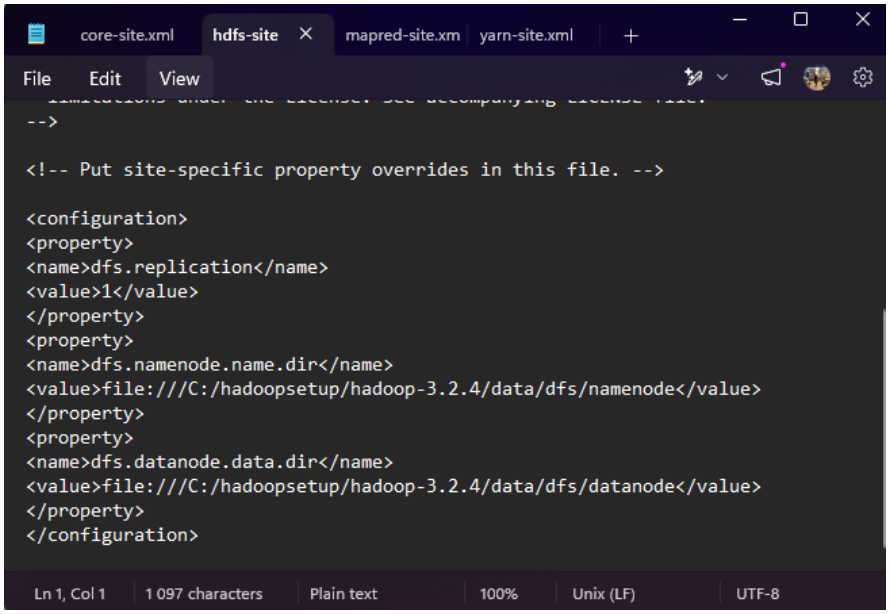


Figure 5: Hdfs-site.xml

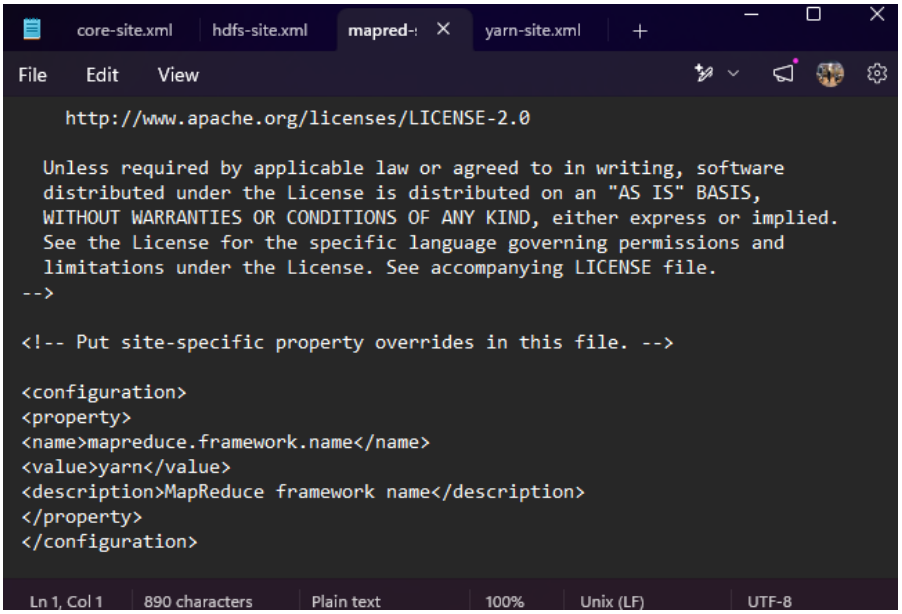


Figure 6: Mapred-site.xml

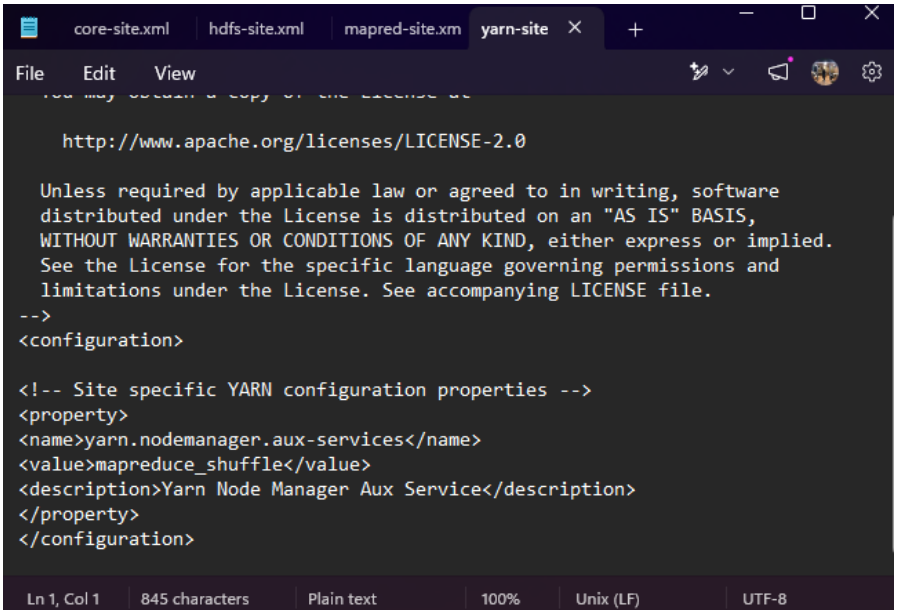


Figure 7: Yarn-site.xml

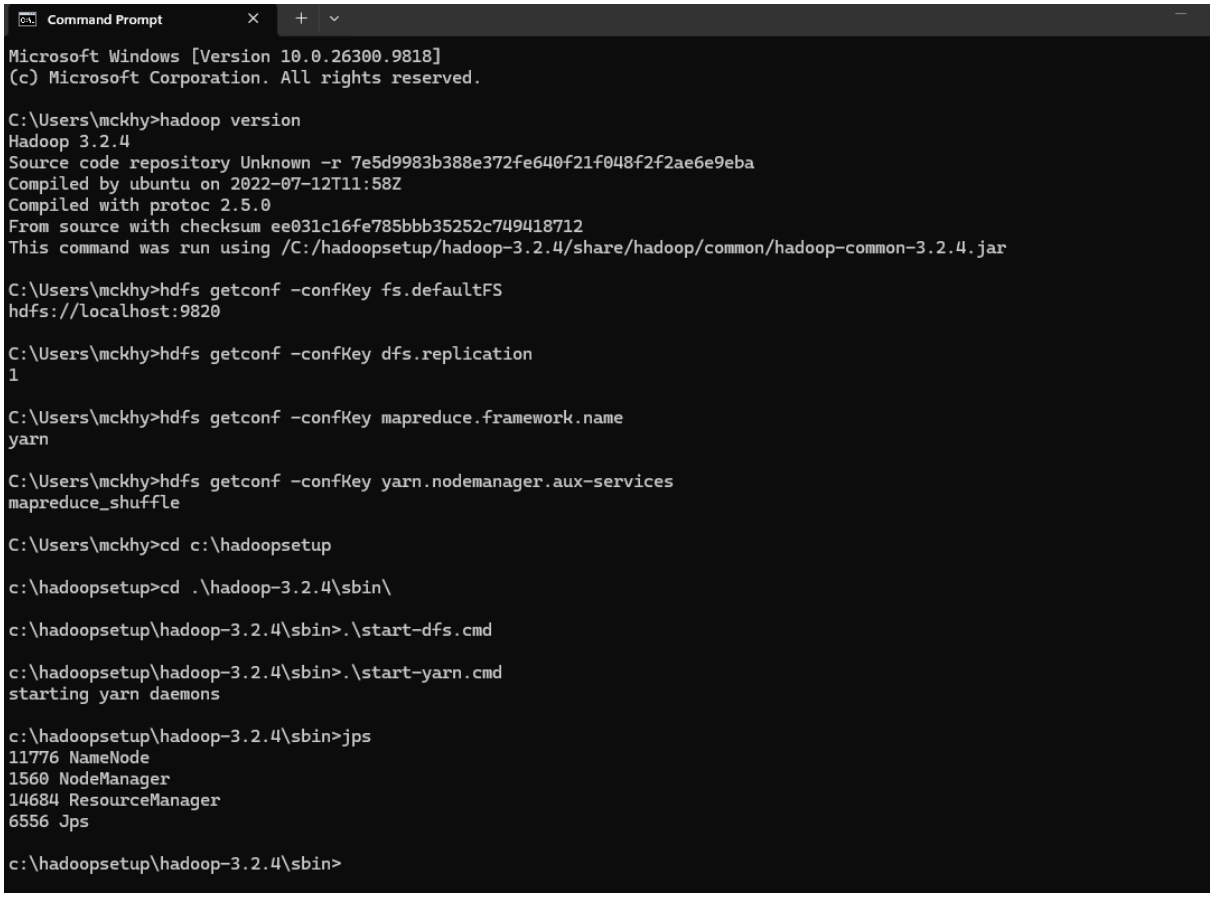


Figure 8: Command Prompt Verification of Hadoop 3.4.2 Configuration

Question 2:

2.1) Detailed explanation of code used to generate the news data frame

The news dataset ("sa_news") was extracted and in the folder contained ten text documents (news_1.txt-news_10.txt). The files were loaded and preprocessed using RStudio.

The question required a data frame to be generated. A data frame containing two columns “File_Name” and “Content” was created using the data.frame() function. Thereafter I used the readLines() function for reading each document, while the paste0() function combined multiple lines into a single text string.

In order to improve the quality and consistency of the text file content, there were several denoising techniques that was applied:

1. Lowercase: Making use of the tolower() function for converting all text into lowercase.
2. Punctuation Removal: The gsub() function aided in removing the punctuation characters.
3. Whitespace Removal: Additionally, the gsub() function replaced repeated whitespace with single spaces.
4. Whitespace Trimming: The trimws() function removed any unnecessary leading and trailing spaces.

Thereafter, the cleaned text was stored within the Content column of the data frame which contained 10 rows and 2 columns. This phase of preprocessing prepares the dataset for further text analysis such as the following LDA question.

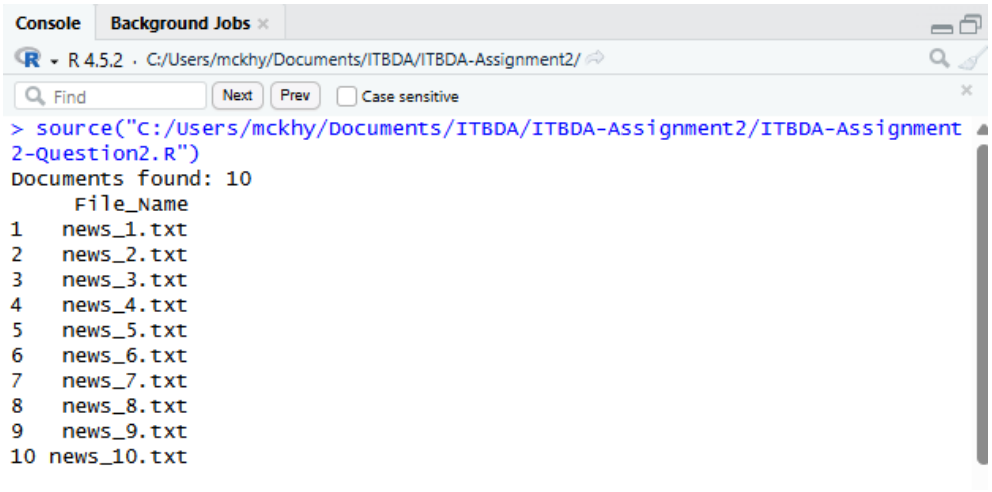
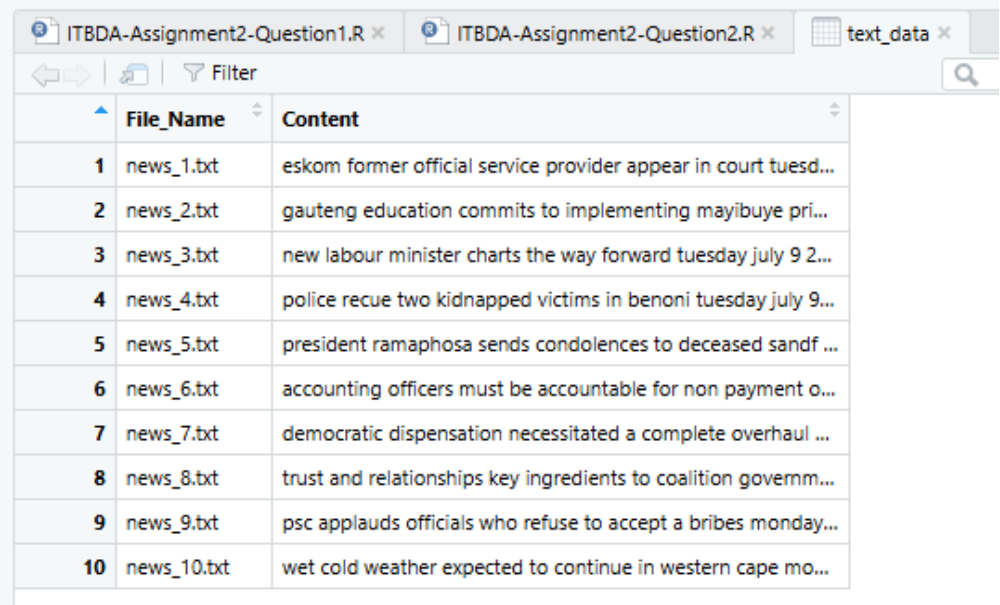


Figure 9: Initial Dataset Loading



	File_Name	Content
1	news_1.txt	eskom former official service provider appear in court tuesd...
2	news_2.txt	gauteng education commits to implementing mayibuye pri...
3	news_3.txt	new labour minister charts the way forward tuesday july 9 2...
4	news_4.txt	police recue two kidnapped victims in benoni tuesday july 9...
5	news_5.txt	president ramaphosa sends condolences to deceased sandf ...
6	news_6.txt	accounting officers must be accountable for non payment o...
7	news_7.txt	democratic dispensation necessitated a complete overhaul ...
8	news_8.txt	trust and relationships key ingredients to coalition governm...
9	news_9.txt	psc applauds officials who refuse to accept a bribes monday...
10	news_10.txt	wet cold weather expected to continue in western cape mo...

Figure 10: Data Frame of the documents

2.2) Topic Modelling Using Latent Dirichlet Allocation (LDA)

LDA was implemented in RStudio in order to identify underlying topics within the ten preprocessed text documents.

The use of the tm package was utilised to create a text corpus, remove common English stop words and to create a Document Term Matric (DTM). The topicmodels was then used to implement the LDA. The LDA was configured to identify 3 topics (n = 3) and then extract the top 5 words (k = 5) that is associated with each topic. A fixed random seed of 123 was also specified to ensure reproducibility. The posterior() function aided in retrieving the word probability and document topic distributions.

Below and the results and interpretation of the LDA model and its findings on the topic word distributions:

Topic	Top 5 Words	Probabilities
Topic 1	Said, public, south, eskom, police	0.0225, 0.0175, 0.0163, 0.0138, 0.0138
Topic 2	Department, unemployment, said, meth, priority	0.0277, 0.0148, 0.0129, 0.0129, 0.0129
Topic 3	Said, government, invoices, national, days	0.0193, 0.0183, 0.0172, 0.0132, 0.0132

As per the results, Topic 1 could possibly focusses on South African public affairs within both electricity (eskom) and law (police).

Topic 2 top five words suggest themes relating to departmental matters, unemployment and the public.

Topic 3 could imply relation to the government administration, particularly within invoices and nation matters.

As per the results, it is worth noting that the word “said” appears in all three topics top 5 words. Although this may be a common word amongst the topics, the word itself makes the topics less distinctive.

The document topic distribution indicates the following dominant topics within the text documents:

Dominant Topic	Documents	Num. of Documents
Topic 1	news_1, news_4, news_5, news_7, news_9	5
Topic 2	news_2, news_3	2
Topic 3	news_6, news_8, news_10	3
Total		10

As per the above results, Topic 1 displayed the most consistent dominant topic which appeared in 50%/five out of the 10 documents. Following Topic 1 is Topic 3 with 30%, while Topic 2 scored the least with only 20%.

Each document returned a dominant topic probability above 99.9% which can imply that the fitted model assigned almost all of its estimated topic proportion to a single topic. Although this occurrence is accurate, it should also be interpreted cautiously as the dataset only has ten documents.

Overall, the LDA application was successfully in generating three topics and also identifying the five most probable words.

ITBDA-Assignment2-Question2.R | text_data | top_words

Filter

	Topic	Word	Probability
1	1	said	0.0225
2	1	public	0.0175
3	1	south	0.0163
4	1	eskom	0.0138
5	1	police	0.0138
6	2	department	0.0277
7	2	unemployment	0.0148
8	2	said	0.0129
9	2	meth	0.0129
10	2	priority	0.0129
11	3	said	0.0193
12	3	government	0.0183
13	3	invoices	0.0172
14	3	national	0.0132
15	3	days	0.0132

Figure 11: Top 5 words

ITBDA-Assignment2-Question2.R | text_data | top_words | document_topics

Filter

	File_Name	Topic_1	Topic_2	Topic_3
1	news_1.txt	0.9997	0.0002	0.0002
2	news_2.txt	0.0001	0.9997	0.0001
3	news_3.txt	0.0001	0.9999	0.0001
4	news_4.txt	0.9997	0.0002	0.0002
5	news_5.txt	0.9997	0.0002	0.0002
6	news_6.txt	0.0001	0.0001	0.9998
7	news_7.txt	0.9997	0.0001	0.0001
8	news_8.txt	0.0001	0.0001	0.9999
9	news_9.txt	0.9998	0.0001	0.0001
10	news_10.txt	0.0001	0.0001	0.9998

Figure 12: Topic Probability

2.3) The 2 visualizations that I have chosen is a Bar graph of Top Words per Topic and a Heat Map.

1. The Bar Graph Plot displays the top words in each topic and their probabilities. Additionally, which words are most important in each topic.

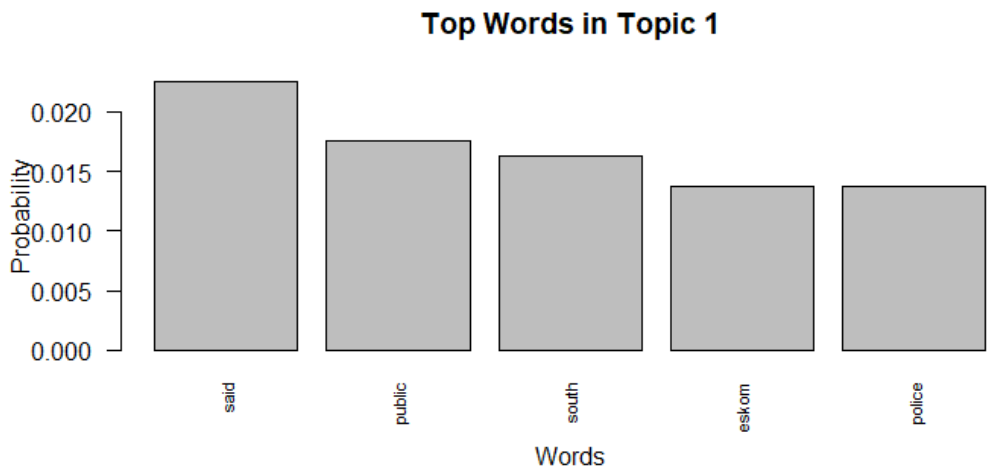


Figure 13: Topic 1 Bar Graph

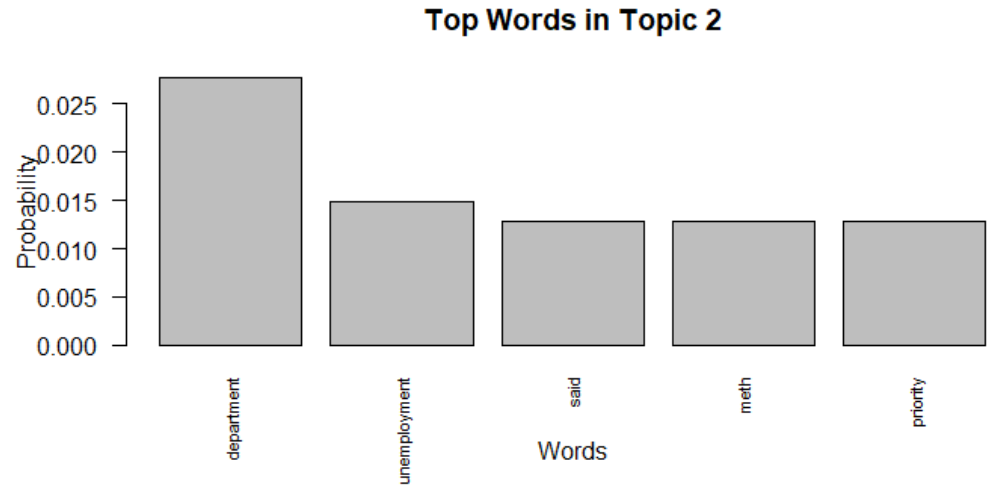


Figure 14: Topic 2 Bar Graph

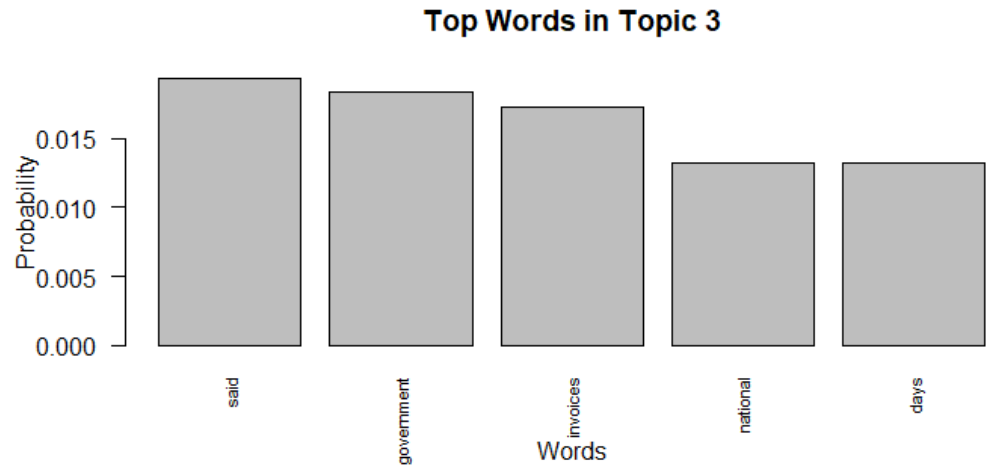


Figure 15: Topic 3 Bar Graph

2. A heat map was created to visualise keyword probabilities distributions across all three topics. The heat map makes use of colour for representing the probability of each keyword within a particular topic. This will also

allow for a comparison between the topics. The heatmap also allows a keyword such as “said” to be compared across multiple topics.

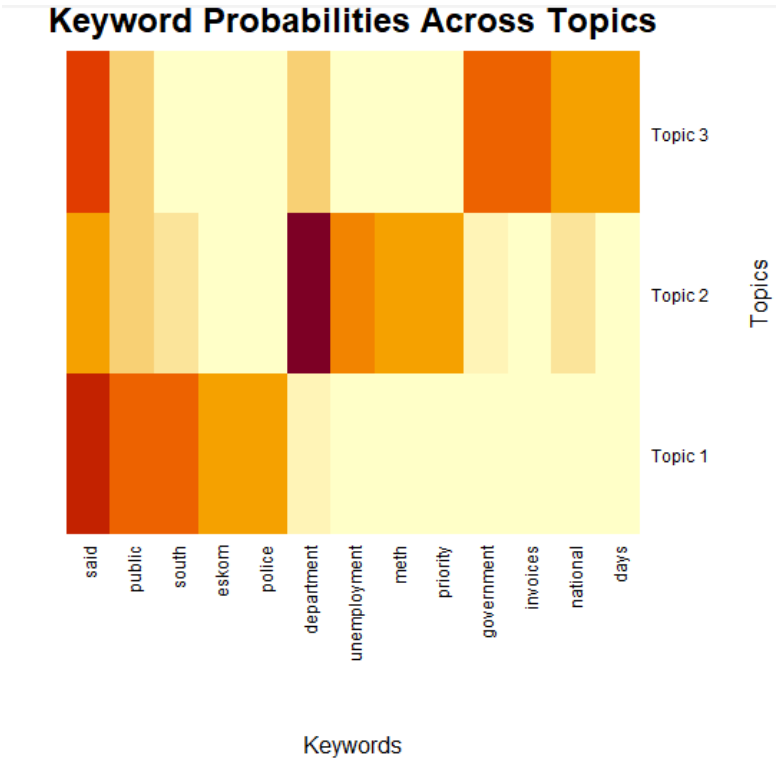


Figure 16: Heatmap Topic Word Distribution

Question 3:

3.1) Ethical webscraping and text extraction

The website that I have chosen with compliance of web scraping is: Zytes Web Scraping Sandbox (toscrape), specifically the Quotes of famous people.

Link: https://toscrape.com/ and https://quotes.toscrape.com/

The website that I have selected for this question is Quotes to Scrape. The reason for choosing this website is because it forms part of an educational web scraping sandbox and it is specifically designed for practising automated data extraction. Additionally, this website is commonly used for such practises as was built exactly for the use of being scraped, meaning I have direct consent to extract the websites data. The website also contains public domain quotes which means that I do not have to be concerned about Intellectual Property (IP) or data privacy laws.

Regarding the data analysis, the scraping was limited to the websites first page which focussed on accessible quotes from the respective authors. This approach ensured following responsible web scraping practices by limiting the amount of information collected and avoiding any unwanted requests from the website.

The text extraction and cleaning process follows the installation of the rvest package as I have used RStudio. The rvest package enabled the retrieval and processing of the webpages HTML content. The read_html() function was utilised to access the website, and the html_elements() function was used to identify the HTML

elements that contained quotation text and author names. The html_text2() function was then applied for extracting readable text content from the selected HTML elements.

After extraction, the trimws() function was the only preprocessing function used at this point to remove any unnecessary leading and trailing white spaces.

Thereafter, the extracted information was formed into a an R data frame using the data.frame() function. The resulting data frame displayed 10 rows and 2 variables/columns. These are “Quotes” and “Authors”. The observations represented the individual quotes and the two variables stored the quotation text and its respective author names. The str() and dim() function were used to identify and verify the structure of the data frame.

Overall, this question tested how ethical web scraping techniques can be implemented in an R environment for retrieving public text which was cleaned and storing the information in a structured data format.

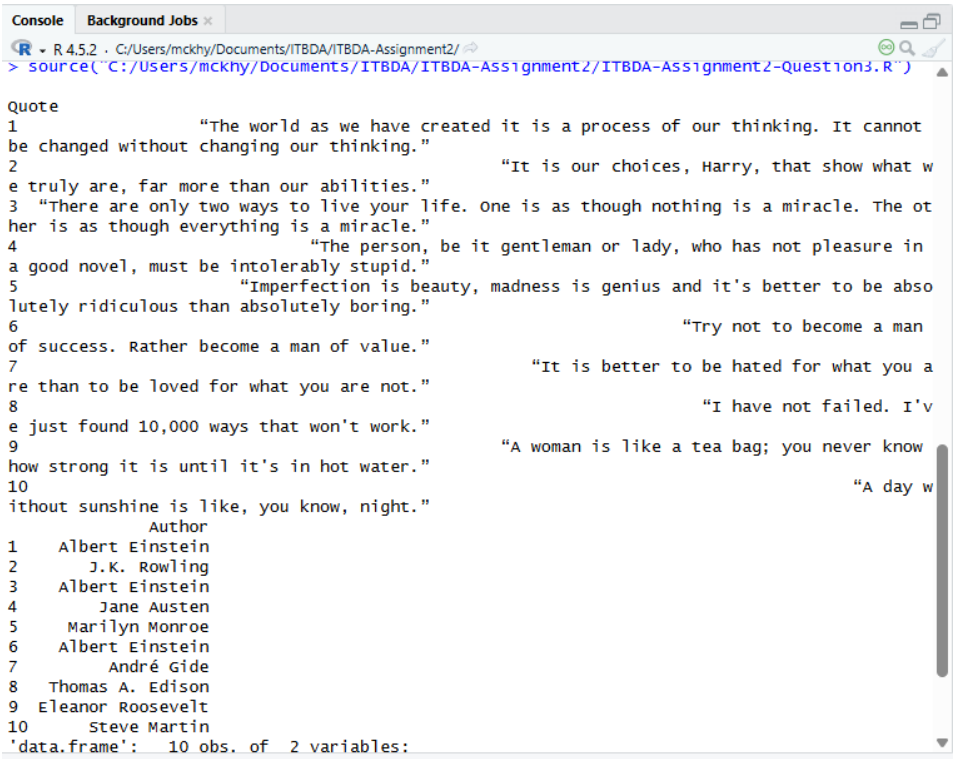


Figure 17: Quote and Author extraction

ITBDA-Assignment2-Question3.R × quote_dataframe ×

Filter

	Quote	Author
1	"The world as we have created it is a process of our thinking...	Albert Einstein
2	"It is our choices, Harry, that show what we truly are, far mor...	J.K. Rowling
3	"There are only two ways to live your life. One is as though ...	Albert Einstein
4	"The person, be it gentleman or lady, who has not pleasure i...	Jane Austen
5	"Imperfection is beauty, madness is genius and it's better to...	Marilyn Monroe
6	"Try not to become a man of success. Rather become a man...	Albert Einstein
7	"It is better to be hated for what you are than to be loved fo...	André Gide
8	"I have not failed. I've just found 10,000 ways that won't wor...	Thomas A. Edison
9	"A woman is like a tea bag; you never know how strong it is ...	Eleanor Roosevelt
10	"A day without sunshine is like, you know, night."	Steve Martin

Figure 18: Data frame of Quotes and Authors

3.2) The textual data extracted from the Quotes to Scrape website went through several preprocessing techniques for the purpose of removing any information and prepare the content for further analysis. The following techniques were applied:

1. Lowercase: Using tolower() function for ensuring consistent text formatting.
2. URL removal: The gsub() function aided in the removal of any URL's present.
3. Number removal: The gsub() function was used to remove any numerical values present.
4. Punctuation removal: The gsub() function was used for removing any punctuation present.
5. Isolated letter removal: The gsub() function ensured any single letter after punctuation removal such as 's' for it's.
5. Whitespace removal: The gsub() function was used to remove any whitespaces between text and trimws() function which was already used previously removes any spaces from start to end of the text.
6. Stop word removal: setdiff() function excludes any negative words from the stop word list such as "no" or "never".

As per the website at face value, I cannot see any links however as a precaution I have included the technique as a standard preprocessing practise.

A basic loop was implemented in order to process each quotes separately, separating the text into individual words, removal of the selected stop words, and combining the remaining words into a cleaned text format. The preprocessed quotations were recorded and stored in a new column called Cleaned_Quote. The new column is an addition to the existing data frame so therefore there are still 10 rows/observations, but now there are 3 columns.

ITBDA-Assignment2-Question3.R | quote_dataframe

Filter

	Quote	Author	Cleaned_Quote
1	"The world as we have created it is a process of our thinking...	Albert Einstein	world created process thinking changed without changing t...
2	"It is our choices, Harry, that show what we truly are, far mor...	J.K. Rowling	choices harry show truly far abilities
3	"There are only two ways to live your life. One is as though ...	Albert Einstein	two ways live life one though nothing miracle though everyt...
4	"The person, be it gentleman or lady, who has not pleasure i...	Jane Austen	person gentleman lady not pleasure good novel must intole...
5	"Imperfection is beauty, madness is genius and it's better to...	Marilyn Monroe	imperfection beauty madness genius s better absolutely ridi...
6	"Try not to become a man of success. Rather become a man...	Albert Einstein	try not become man success rather become man value
7	"It is better to be hated for what you are than to be loved fo...	André Gide	better hated loved not
8	"I have not failed. I've just found 10,000 ways that won't wor...	Thomas A. Edison	not failed ve just found ways won t work
9	"A woman is like a tea bag; you never know how strong it is ...	Eleanor Roosevelt	woman like tea bag never know strong s hot water
10	"A day without sunshine is like, you know, night."	Steve Martin	day without sunshine like know night

Figure 19: Cleaned Quote Column

3.3)

Introduction

The two text summarisation techniques that I chose are Frequency Based Extractive Summarisation and TF-IDF Centroid-Based Extractive Summarisation. The chose text summarisations were applied to the Quotes Web Scraping data extracted to identify representative quotations. Both the techniques were applied to the 10 preprocessed quotations. Each method used selected the three highest scoring frequencies.

Console | Background Jobs

R 4.5.2 · C:/Users/mckhy/Documents/ITBDA/ITBDA-Assignment2/

```
> print(top_words)
       not absolutely     become     better       know       like        man    miracle
         4          2          2          2          2          2          2          2
  thinking     though
         2          2
>
```

Figure 20: Top Words for Summarisation

Frequency-Based Extractive Summarisation

The first chosen technique did involve calculating word frequencies within the extracted data. The quotations were separated into individual words and their occurrences were recorded to identify the frequency of each term. The quotations were then ranked respectively to their scores, and the three highest scores were selected.

Results

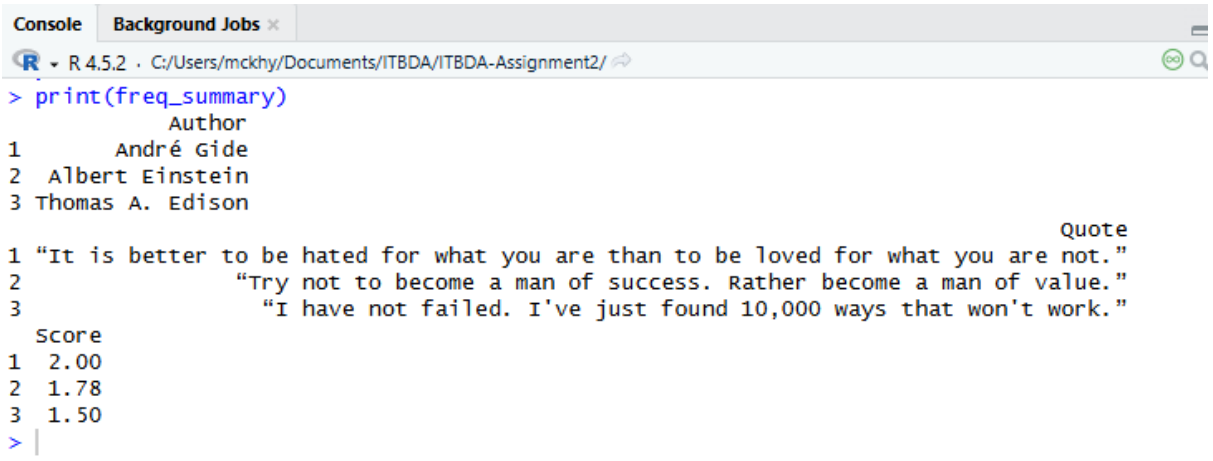


Figure 21: Frequency-Based Extractive Summarisation

As per the results, the quote by Andre Gide score the highest frequency score of 2.00. This means that the quotes words occurred more throughout the extracted textual data. Following this result is Albert Einstein with a score of 1.78, and Thomas Edison with a score of 1.50. These results show how frequency based summarisation makes commonly occurring vocabulary a priority.

The second technique is Term Frequency-Inverse Document Frequency, also known as TF-IDF. This technique assigned weights to words that are based on their frequency within individual quotations and their distribution within the extracted data. Additionally, a centroid calculates the average TF-IDF values, while the cosine similarity shows how closely each quote matched the data’s average.

Results

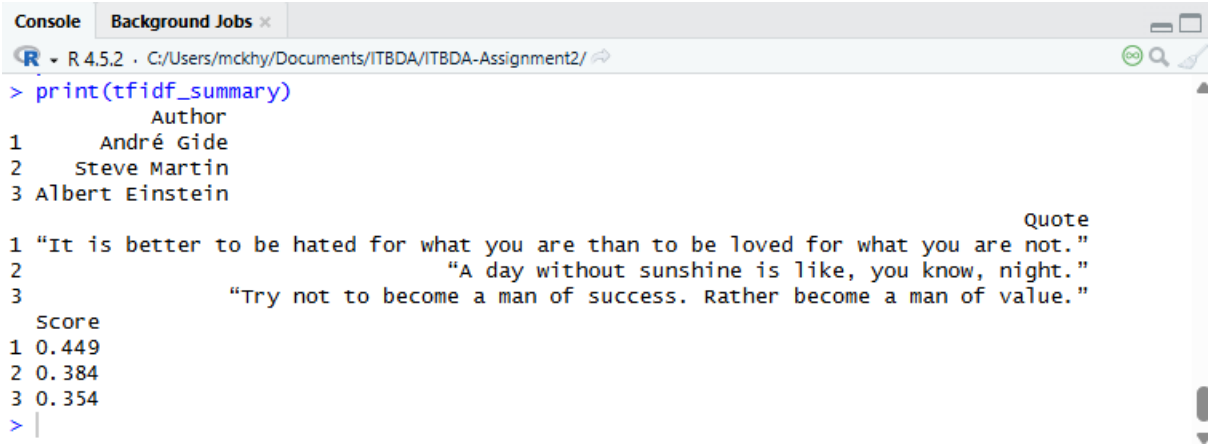


Figure 22: TF-IDF Summarisation

As per the results above, Andre Gide again obtained the highest score with a score of 0.449. This was followed by Steve Martin with a score of 0.384 and Albert Einstein with a score of 0.354. Overall, this techniques finding displays how TF-IDF selects quotation based on their similarity to their overall vocabulary representation. Rather than relying on word frequency on its own.

Visualisations

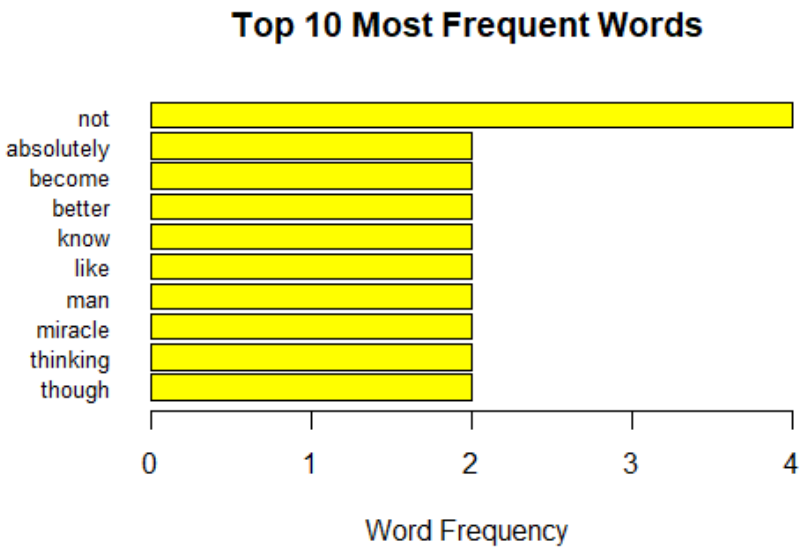


Figure 23: Top 10 Word Frequency

As per the graph above, a horizontal bar graph was chosen to represent the 10 most frequently occurring words within the cleaned data. As per the results, only the word "not" (a negative word) appears 4 times while the rest of the words occur twice. The higher value of the word "not" may be due to the preservation of the word within the data preprocessing steps as without the word, the quotation may become less readable. Overall, the visualisation provides a distinct but meaningful overview on the reoccurring terms and supports the interpretations made of the frequency based summarisation results.

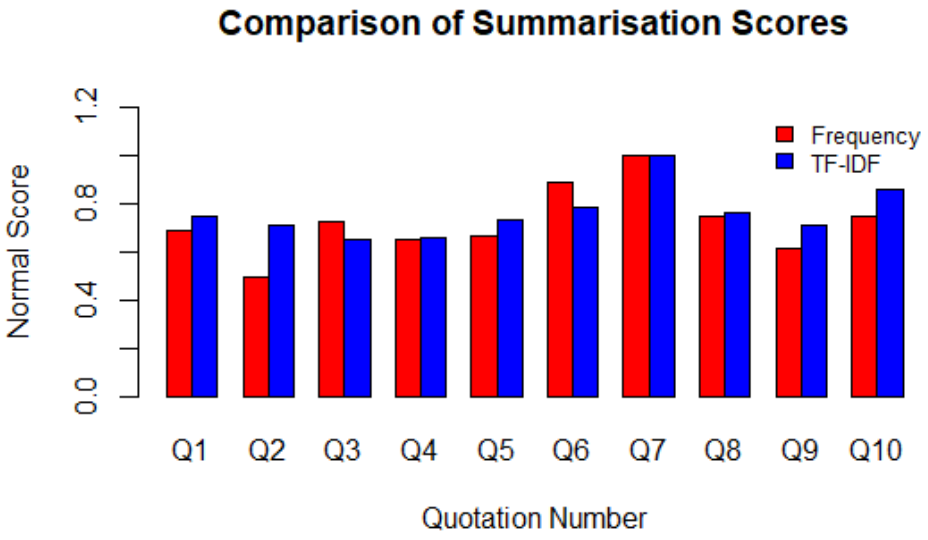


Figure 24:Frequency Based and TF-IDF Summarisation Comparison

A grouped bar graph visualisation was used to present and more importantly compare the frequency based and TF-IDF similarity scores for the quotations. The scores were first normalised to the maximum score in each of the summarisation techniques to support visual comparison.

As per the results, quotations by Andre Gide and Albert Einstein were selected, however the frequency based

technique also selected Thomas Edison. Whereas the TF-IDF technique selected Steve Martin instead.

Discussion

Synthesizing the summarisation task, the results showed that the two techniques utilised both produced similarities and differences in the selected quotes. Frequency Based summarisation made quotations containing words that occurred frequently a priority, while TF-IDF summarisation prioritised quotes whose weighted vocabulary was similar in comparison to the entire extracted data. Due to the data extracted only containing 10 quotations, the results require to be interpreted as exploratory task selections instead of comprehensive summaries.

Question 4:

4.1) Web Scraping and Regex analysis

For this task, I have made use of the exact same website as per the previous question. However, I did not use the "Quotes" section but rather the "Books" section. This is accessible via (https://books.toscrape.com). This website is educational and is designed specifically for practising web scraping techniques.

The object of this tasks was to extract production information, identify and analyse incomplete or inconsistent formatted content. Thereafter, to apply regular expressions to form structured information that is suitable for analysis.

Within the data analysis, the rvest library was utilised. This aided in the extraction of 20 book records from the selected website. The extracted information included the book titles, the full book titles, prices with the currency symbols, and availability. The raw values were kept as is to allow for a comparison between the original and processed data.

Now utilising regex methods, the grepl() function was used to identify any book titles that contained an ellipses (…) in the books title as this meant that the books title was already abbreviated. Following this was the gsub() function. This function aided in the removal of currency symbols and other non-numeric related characters within the extracted prices. The recorded values where then converted into a numeric format using as.numeric()

The next function used was trimws() which was used for removing any whitespaces from the availability information. Below is the final data frame of the extracted text:

ITBDA-Assignment2-Question4.R × book_data ×

Filter

	Display_Title	Full_Title	Raw_Price	Availability	Title_Incomplete	Price
1	A Light in the ...	A Light in the Attic	£51.77	In stock	TRUE	51.77
2	Tipping the Velvet	Tipping the Velvet	£53.74	In stock	FALSE	53.74
3	Soumission	Soumission	£50.10	In stock	FALSE	50.10
4	Sharp Objects	Sharp Objects	£47.82	In stock	FALSE	47.82
5	Sapiens: A Brief History ...	Sapiens: A Brief History of Humankind	£54.23	In stock	TRUE	54.23
6	The Requiem Red	The Requiem Red	£22.65	In stock	FALSE	22.65
7	The Dirty Little Secrets ...	The Dirty Little Secrets of Getting Your Dream Job	£33.34	In stock	TRUE	33.34
8	The Coming Woman: A ...	The Coming Woman: A Novel Based on the Life of the Infam...	£17.93	In stock	TRUE	17.93
9	The Boys in the ...	The Boys in the Boat: Nine Americans and Their Epic Quest f...	£22.60	In stock	TRUE	22.60
10	The Black Maria	The Black Maria	£52.15	In stock	FALSE	52.15
11	Starving Hearts (Triangular Trade ...	Starving Hearts (Triangular Trade Trilogy, #1)	£13.99	In stock	TRUE	13.99
12	Shakespeare's Sonnets	Shakespeare's Sonnets	£20.66	In stock	FALSE	20.66
13	Set Me Free	Set Me Free	£17.46	In stock	FALSE	17.46
14	Scott Pilgrim's Precious Little ...	Scott Pilgrim's Precious Little Life (Scott Pilgrim #1)	£52.29	In stock	TRUE	52.29
15	Rip it Up and ...	Rip it Up and Start Again	£35.02	In stock	TRUE	35.02
16	Our Band Could Be ...	Our Band Could Be Your Life: Scenes from the American Indi...	£57.25	In stock	TRUE	57.25
17	Olio	Olio	£23.88	In stock	FALSE	23.88
18	Mesaerion: The Best Science ...	Mesaerion: The Best Science Fiction Stories 1800-1849	£37.59	In stock	TRUE	37.59
19	Libertarianism for Beginners	Libertarianism for Beginners	£51.33	In stock	FALSE	51.33
20	It's Only the Himalayas	It's Only the Himalayas	£45.17	In stock	FALSE	45.17

Showing 1 to 20 of 20 entries, 6 total columns

Figure 25: Extracted Book Data

As per the results above, there are a total of 20 book records that have been successfully extracted from the website. Of these 20 records, 10 (50%) of the book titles under column “Display_Title” had ellipses which were identified by using regular expressions and was compared against the complete title of the books.

The raw priced of the book contained the Pound (£) currency, which required further processing before numerical analysis. Therefore as shown, the “Price” column contains the prices without the symbol.

4.2) Data Cleaning, Denoising and Missing Value Imputation

The scraped books were cleaned and denoised for the purpose of improving its consistency and suitability for further analysis. The abbreviated book titles were replaced with the equivalent column. Additionally, the whitespaces was removed from the text. Any meaningful numerical values and special characters such as number, hashtags, or brackets were retained as part of the book title to preserve the original information.

Missing numerical values were checked using the is.na() function. As part of this imputation process conditional median was implemented where necessary however there were no missing prices that were identified so therefore no need for numerical imputation. As per below figure shows the results of the cleaned data frame:

ITBDA-Assignment2-Question4.R | book_data | cleaned_data

Filter

	Title	Price	Availability
1	A Light in the Attic	51.77	In stock
2	Tipping the Velvet	53.74	In stock
3	Soumission	50.10	In stock
4	Sharp Objects	47.82	In stock
5	Sapiens: A Brief History of Humankind	54.23	In stock
6	The Requiem Red	22.65	In stock
7	The Dirty Little Secrets of Getting Your Dream Job	33.34	In stock
8	The Coming Woman: A Novel Based on the Life of the Infamous Feminist, Victoria Woodhull	17.93	In stock
9	The Boys in the Boat: Nine Americans and Their Epic Quest for Gold at the 1936 Berlin Olympics	22.60	In stock
10	The Black Maria	52.15	In stock
11	Starving Hearts (Triangular Trade Trilogy, #1)	13.99	In stock
12	Shakespeare's Sonnets	20.66	In stock
13	Set Me Free	17.46	In stock
14	Scott Pilgrim's Precious Little Life (Scott Pilgrim #1)	52.29	In stock
15	Rip it Up and Start Again	35.02	In stock
16	Our Band Could Be Your Life: Scenes from the American Indie Underground, 1981-1991	57.25	In stock
17	Olio	23.88	In stock
18	Mesaerion: The Best Science Fiction Stories 1800-1849	37.59	In stock
19	Libertarianism for Beginners	51.33	In stock
20	It's Only the Himalayas	45.17	In stock

Showing 1 to 20 of 20 entries, 3 total columns

Figure 26: Cleaned Extracted Book Data

As per the results, the data still contains 20 records, but now only 3 columns. There are Title, Price, and Availability. Finally, the cleaned data was exported as a CSV file “cleaned_books.csv” using the write.csv() function.

4.3) PostgreSQL Data Import and SQL based analysis

This task required PostgreSQL to be used. It was used through pgAdmin 4 for creating a relational table called books. Within the table contained three attributes: title, price, and availability. The previously created CSV file was successfully imported into the created table which contained 20 book records as per below screenshot.

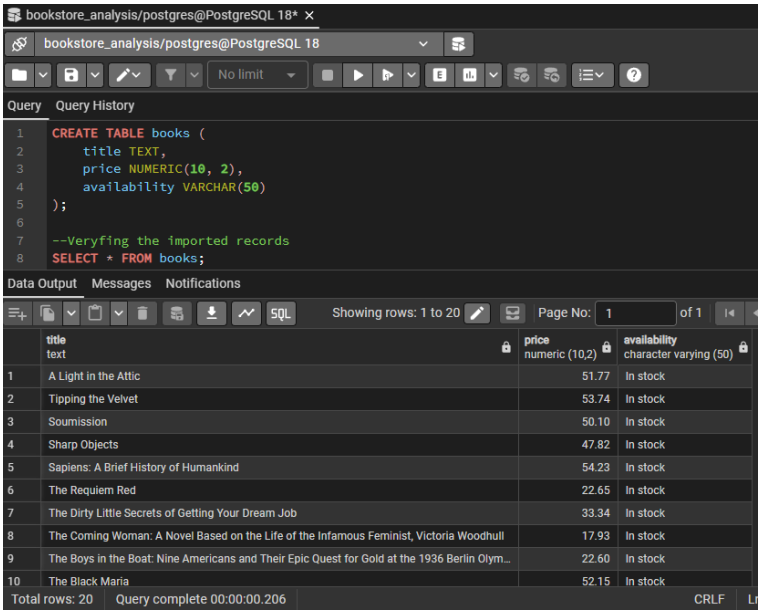


Figure 27: SQL Table Creation and CVS Import

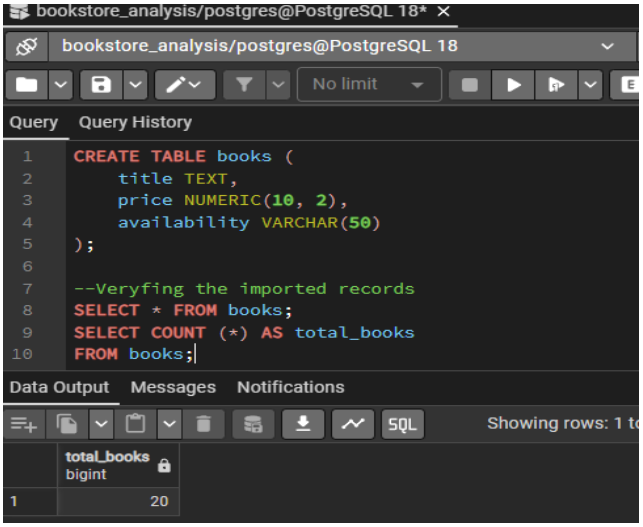


Figure 28: Confirming Book Records

A SELECT statement and the COUNT() function was used to confirm the imported information.

Investigation 1: Overall Book Price Analysis

The first SQL investigation examined the overall pricing characteristics of the bookstore dataset. The following aggregate functions such as COUNT(), AVG(), MIN(), MAX() were used for determining the total number of books, the average price, the lowest and the highest prices.

As per the results, the average price was 38.05. Subsequently, the lowest price recorded was 13.99 while the highest was 57.25. This could mean that perhaps based on the pricing, the author who wrote the books were more prestigious and a bestseller.

Investigation 2: Book Price Distribution

The second investigation analysed the distribution of books across three pricing categories: Budget, Mid-Range, and Premium. Each had a set value of below £25, between £25 – £45, and above £45 respectively.

This supports the first investigation completed as some books may be more sought after than other. As per the results, the lowest distributed books ranged between £25 – £45 (Mid-Range), while the £45 books (Premium) was the most distributed. The below £25 (Budget) books were second with only 7 books distributed in total.

Overall, these simple but effective investigations demonstrate how SQL can be used efficiently from a R-created CSV file for summarising numerical information and identifying the number of books within each category.

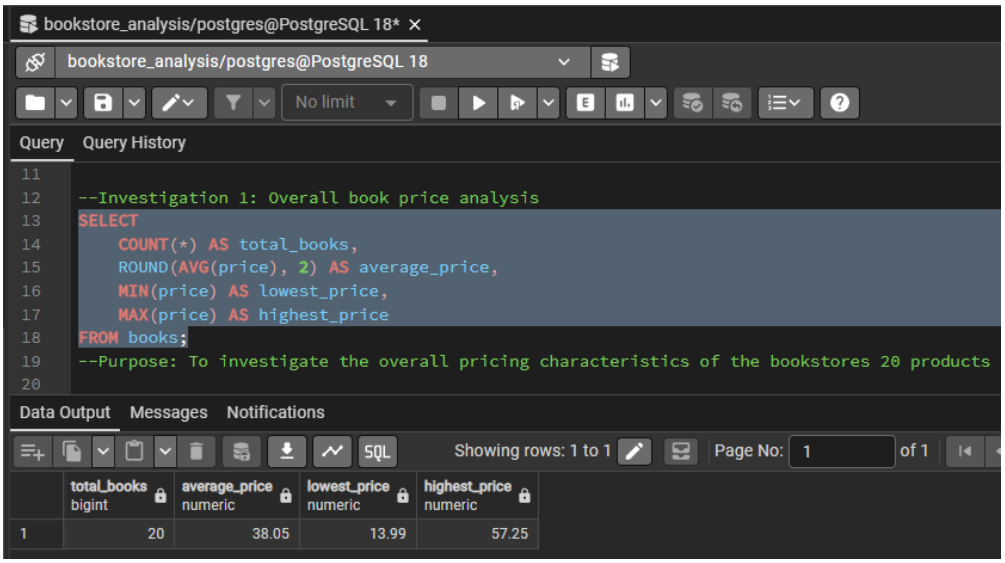


Figure 29: Investigation 1- Overall Book Price Analysis

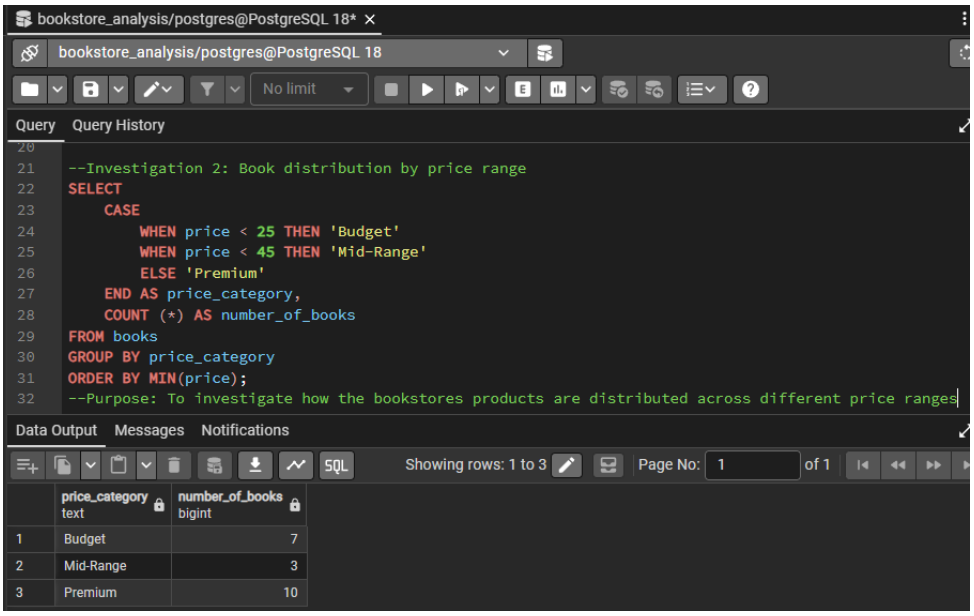


Figure 30: Investigation 2- Book Distribution by Price Range

4.4) Github link: https://github.com/MckhyleGanas/ITBDA_Assignment